Dynamically Hardening Linux Devices on the Network
HeavilyArmedPenguins

Jackson Dodd, Jamin Hanna, Samuel LaVey

Introduction

Heavily Armed Penguins, or HAP for short, is a web-based software application designed to make hardening a device as convenient and accessible as possible. Cybersecurity is becoming a crucial part of the modern world, as information technology has become more ubiquitous than ever. Securing data, finances, trade secrets, and intellectual property is critical as attackers continue to become more sophisticated and penetration tools become more usable by the public.

Over the course of the project, due to time constraints, the scope of this project was scaled down. Rather than act as a complete software suite for cybersecurity on a network, HAP seeks to provide a user with a collection of capabilities that can be used to harden RHEL operating systems. We decided to focus on Linux operating systems due to their familiarity and structure. RHEL was decided as a primary candidate for development due to its popularity.

Overall, and despite the project being scaled down, we were able to accomplish our main goal: to build a complete utility that allows a user to secure their system using a web interface.

Customer Value

3/8/24 - Narrowing of scope in terms of capability and operating systems. We did this due to time constraints both in terms of the semester and the work demanded by other classes. We decided to focus on developing capabilities to harden RHEL, specifically building off of the red hat STIG[1]. Even if we couldn't build the entire STIG, we were able to build out the most crucial capabilities called for. The customer value proposition would be that users who are unfamiliar with defensive cybersecurity techniques, but have some technical knowledge, could use this technology to harden their networks. Our technology works at scale, can be deployed to any size network, and will execute tasks sequentially.

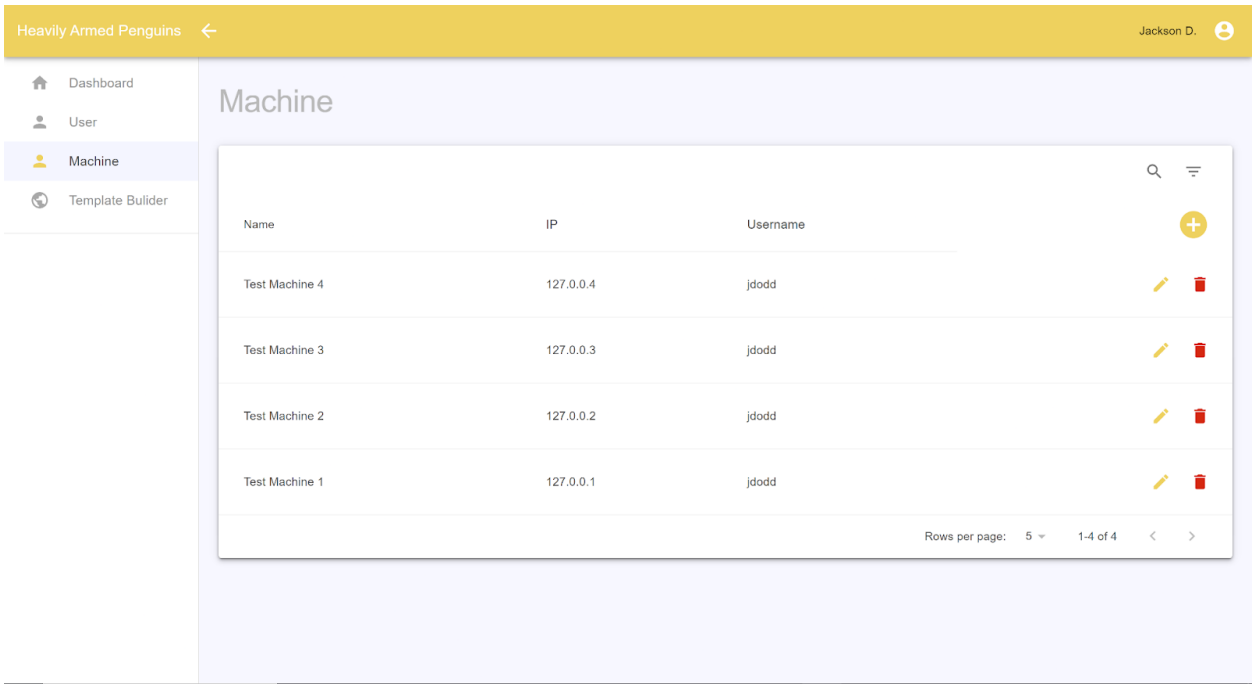
Technology

I. User Interface (UI)

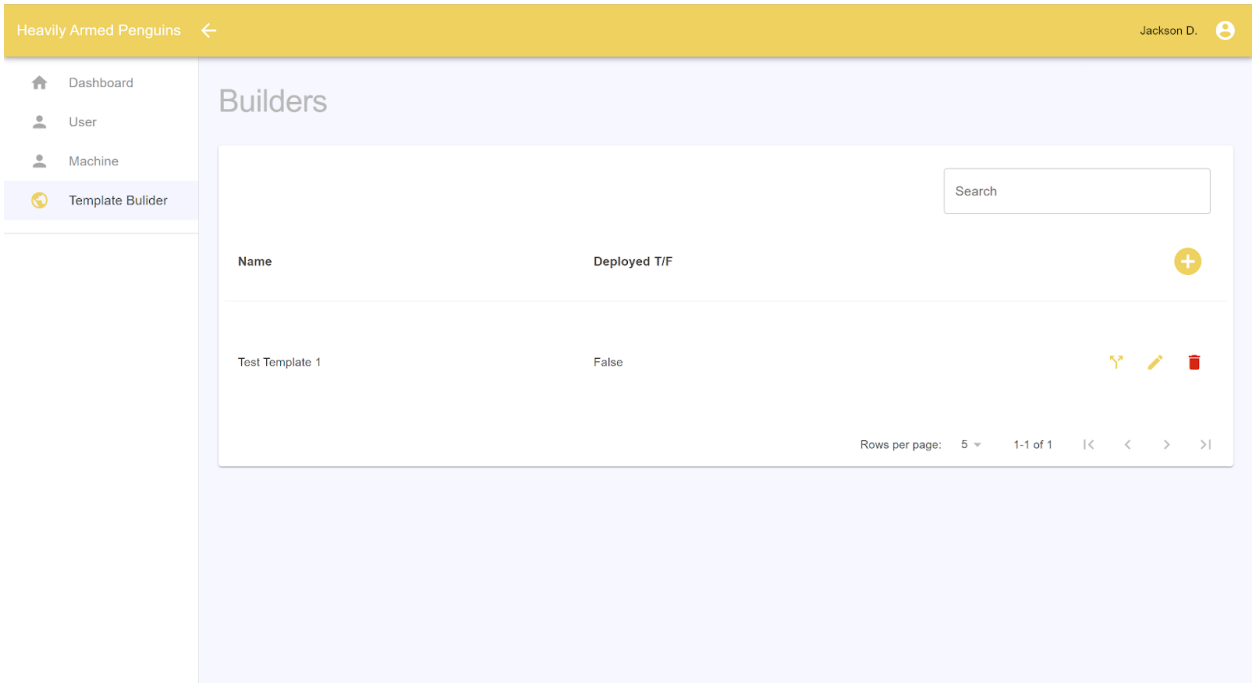
Our user interface consists of a MERN stack web server, which requires a login to authenticate to for security purposes. After logging into the web server, a user will arrive at the dashboard page, pictured here.



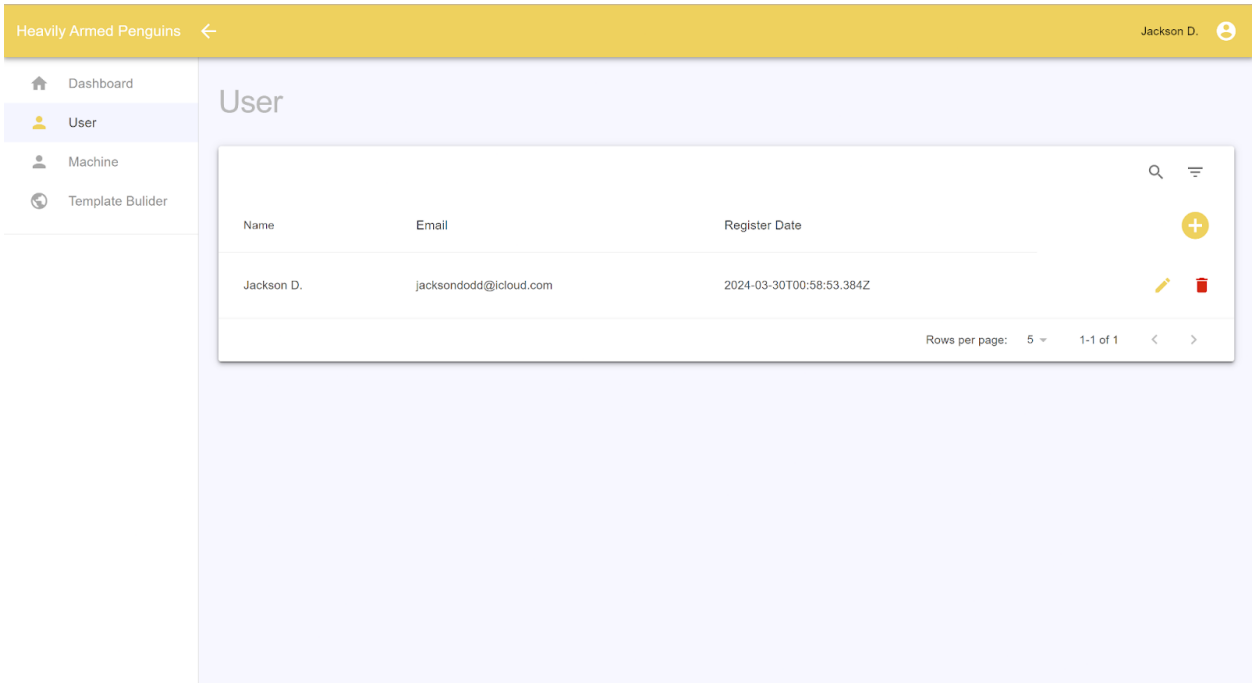
- From the dashboard, a user can easily navigate to the ‘Machine’ tab, which contains buttons to create, edit, and delete machines. It also displays a list of all reachable machine objects, their IP addresses, and their usernames.



- The ‘Template Builder’ tab has a create template button which allows users to construct a security template for deployment on remote machines. It also has buttons to delete and edit templates. Additionally, there's a deployment button next to each template which allows users to deploy that template on a list of machines they specify.



- There is also a ‘User’ tab, pictured here, which allows an admin to administratively add, edit, delete, and see all other users.



II. Modular Capabilities

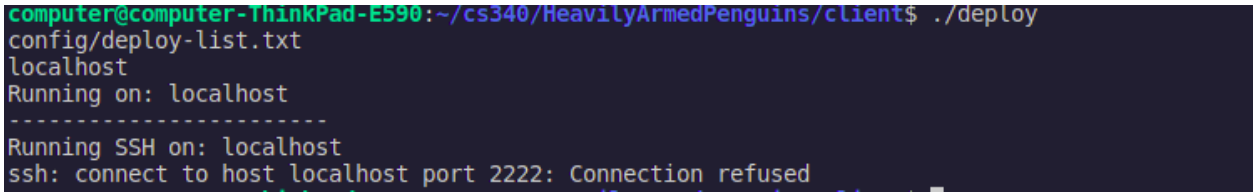
We built out a collection of modular capabilities which were based on the RHEL 9 STIG, which is a standard used by the Department of Defense (DoD) to harden and secure machines. We have a collection of 12 capabilities that we were able to build, test, and deploy to the master branch. While it would have been nice to have more, due to time constraints we had to limit ourselves to only the most critical capabilities. The configuration of our capabilities would allow for easy expansion.

Testing a capability was done according to the following approach. A bash script would be written that was able to autonomously perform the STIG hardening function. This script was directly tested on a virtual machine, and if successful a call was made in our configuration translation program to add the script to our master script at the end.
Pictures of that process are included below.

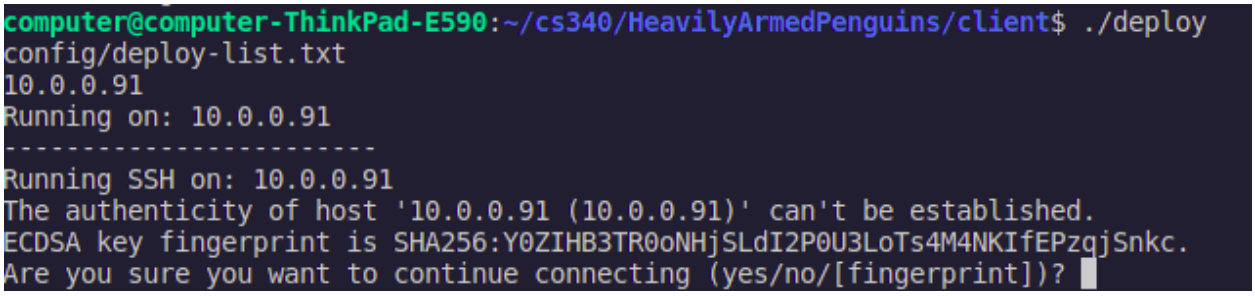
III. Deployment Program

The deployment program was a C++ script that was written to deploy the script that will be built by the configuration translation script on any number of specified machines. It takes in two files, reading deploy-list.txt, found under the config directory, and template.txt, found in the template directory. It then runs through this template.txt file, which contains abstracted language commands, through configtranslation.cpp, which acts as an interpreter. Once the master script has been compiled, it is run on all specified remote hosts using the SSH client, sequentially as they are found in the list.

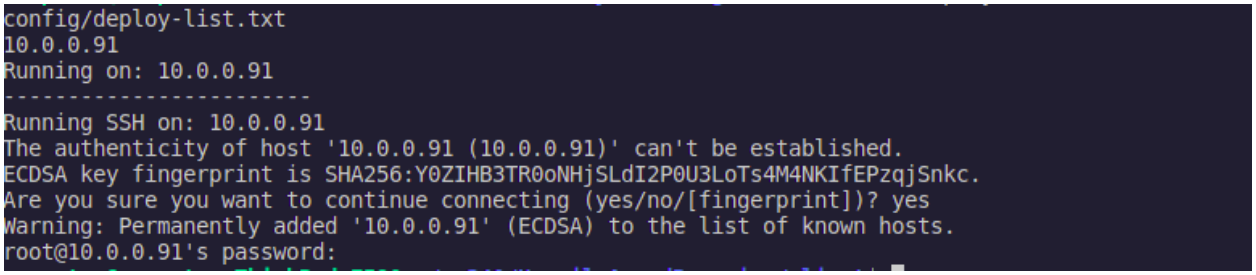
Pictured below are screenshots from our testing of the deployment process. These tests were run on a single machine to demonstrate the successful use of the SSH client for machine modification.



An example of unsuccessful connection. Here, the machine is unreachable.



A successful connection prompt. A prompt will be generated for each machine being accessed.



No error messages means the script was run successfully.

Team

Individual roles were relatively static throughout the course of the project, and are listed below. Sam, the team leader for the project, likely spent the most time developing, as he assumed responsibility for multiple tasks; but both Jackson and Jamin likewise spent ample time developing the functionalities for which they were responsible.

Jackson Dodd

Jackson was primarily responsible for the design and implementation of the front-end webpage as well as the back-end database. This included making a page and database collection for the user data, machine data, and template data. He also implemented the initial communication of information to deploy.

Jamin Hanna

Jamin was primarily responsible for STIG capability development. This included mapping abstracted security configuration commands, developed by Samuel, to bash scripts used to implement the user-specified configurations.

Samuel LaVey

Samuel was primarily responsible for team task coordination, vision, and STIG capability development. He also developed the backend scripts to enable deployment on remote machines.

Project Management

Project goals and schedules had to be continually adjusted as the semester progressed. Due to additional homework from other classes, our group required an additional amount of flexibility with the tasks we had assigned to ourselves. There were also unique technical challenges, especially when it came to the web server refactoring, that took longer than anticipated to resolve. That said, we were unable to accomplish all of our goals on schedule, and due to time constraints had to abandon some of them, but did manage to accomplish those goals which we considered most important.

Reflection

Our planning allowed us to avoid a lot of potential challenges that would have caused unnecessary delays further down the line. Particularly our decision to use pre-existing protocols for network orchestration, rather than inventing our own, saved us a lot of time. SSH is a robust, universally supported, secure protocol that was more than efficient for our purposes. We modeled our orchestration process off of the Ansible scripting language, as their use of abstraction through language saves time and reduces complexity. We wanted to take this concept and extend it to cybersecurity. Using what exists, or modeling off of it, saved time and prevented the type of bugs that often plague the fields of innovation.

Our testing process could have been more organized and was done more in an ad-hoc fashion. This was to put as much time into development as possible. Developers tested their own code individually, and bugs were resolved together post-merge. In future endeavors, a complete lifecycle pipeline would be a necessity, with scripts developed to test each part of the project after each push is made. Time management was done as best as possible given the scholastic circumstances. In the future, future development would be unencumbered by this.

This project was a success, but not in all the ways we had hoped. The framework for orchestration is fully developed and critical capabilities have been deployed. However, a more robust capability set and additional OS support would be desired if we had more time to develop this software.

Sources

[1] https://www.stigviewer.com/stig/red_hat_enterprise_linux_9/